

1. Quality Principles of Total Quality Management (TQM)

Total Quality Management (TQM) is a structured approach to improving product and service quality. It is based on the following principles:

1. **Customer Focus** – The primary goal is to meet or exceed customer expectations by understanding their needs and preferences.
2. **Leadership Commitment** – Strong leadership provides direction, motivation, and a clear vision for quality improvement.
3. **Employee Involvement** – All employees at every level must participate in quality enhancement through training and responsibility-sharing.
4. **Process Approach** – Managing activities as interrelated processes helps achieve efficiency and effectiveness.
5. **Continuous Improvement (Kaizen)** – Organizations should consistently seek ways to improve processes, products, and services.
6. **Data-Driven Decision Making** – Decisions should be based on data analysis, statistics, and performance metrics.
7. **Mutually Beneficial Supplier Relationships** – Developing strong relationships with suppliers ensures a stable supply of high-quality materials and components.
8. **Systematic Approach to Management** – An organization-wide quality management system ensures alignment with strategic goals.

These principles guide organizations in implementing and sustaining a culture of quality.

2. Lifecycle of Quality Improvements

The quality improvement lifecycle consists of six key stages:

1. **Identify Problem Areas**
 - Recognizing quality gaps through customer complaints, testing reports, or defect tracking.
 - Tools: Customer feedback analysis, defect reports, and internal audits.
2. **Analyze Root Causes**
 - Understanding why quality issues occur using tools like Fishbone Diagram (Ishikawa), Pareto Analysis, and the 5 Whys method.
 - Helps in identifying whether the issue is process-related, resource-based, or due to miscommunication.
3. **Plan Solutions**
 - Developing corrective and preventive actions.
 - Strategies include process optimization, automation, or redesigning workflows.
4. **Implement Changes**
 - Applying improvements through revised processes, training programs, or new quality standards.
 - Pilot testing changes before full-scale implementation.
5. **Monitor and Evaluate**

- Measuring improvement using KPIs (defect rate, customer satisfaction, response time).
- Comparing new performance levels against previous benchmarks.

This lifecycle ensures systematic quality enhancement and aligns with Total Quality Management (TQM) and Six Sigma methodologies.

3. Continuous vs. Continual Improvement & PDCA Lifecycle

Continuous Improvement

- Small, incremental improvements made frequently without any fixed end point.
- Common in Agile and DevOps environments where software is updated regularly.
- Example: Frequent software patches to improve security and fix bugs.

Continual Improvement

- Includes periodic reviews and larger-scale changes.
- Focuses on both incremental and breakthrough innovations.
- Example: Adopting a new software development methodology after reviewing performance trends.

PDCA (Plan-Do-Check-Act) Lifecycle

The PDCA cycle is a widely used framework for continuous and continual improvement:

1. **Plan** – Identify a problem, set objectives, and define a strategy for improvement.
2. **Do** – Implement changes on a small scale, such as in a prototype or test environment.
3. **Check** – Analyze data to assess whether improvements are effective.
4. **Act** – Standardize successful changes and apply them at a larger scale.

PDCA helps organizations implement structured quality improvements while reducing risks.

4. Constraints of Product Quality Assessment

Product quality assessment faces several challenges that may impact its accuracy and effectiveness:

1. Cost Constraints

- Comprehensive testing and quality control require significant financial investment.
- Example: Full-scale performance testing may be skipped due to budget limitations.

2. Time Limitations

- Strict deadlines can force teams to reduce testing time, increasing defect risks.
- Example: A software release scheduled for a specific date may limit extensive testing.

3. Resource Availability

- Lack of skilled personnel, testing tools, or infrastructure may hinder effective quality evaluation.
- Example: A company without an automated testing setup may struggle with large-scale performance testing.

4. Subjective Customer Expectations

- Users may have different definitions of "quality," making it difficult to measure success universally.
- Example: One user may expect a minimalist UI, while another prefers feature-rich software.

5. Rapid Technological Changes

- New technologies may render previous quality assessments outdated.
- Example: A web application tested for older browsers may fail in newer browser versions.

Organizations must balance these constraints by prioritizing critical quality aspects and optimizing testing strategies.

5. Definition of Quality & Different Views

Definition of Quality

Quality is the degree to which a product or service meets user expectations, industry standards, and regulatory requirements. It is commonly measured in terms of reliability, performance, and customer satisfaction.

Different Views of Quality

1. Transcendent View

- Quality is an inherent characteristic that is recognized rather than defined.
- Example: Apple products are perceived as high quality due to brand reputation.

2. Product-Based View

- Quality is based on measurable attributes of the product.
- Example: A laptop with higher RAM and faster processors is considered of higher quality.

3. User-Based View

- Quality depends on how well a product meets customer needs.
- Example: A mobile app is of high quality if it has an intuitive UI and no crashes.

4. Manufacturing-Based View

- Quality is achieved through adherence to predefined standards and processes.
- Example: A defect-free software release that meets all functional requirements.

5. Value-Based View

- Quality is evaluated based on performance relative to cost.
- Example: Open-source software offering premium features at no cost is considered high-quality.

Each perspective plays a critical role in software testing and quality assurance.

6. User's Gap vs. Producer's Gap

User's Gap

- The difference between what the user expects and what they actually experience.
- Caused by usability issues, unclear requirements, or misleading marketing.
- Example: A video conferencing app that lacks background noise cancellation despite user expectations.

Producer's Gap

- The difference between what the development team intended to deliver and what was actually built.
- Caused by misinterpretation of requirements, communication failures, or resource limitations.
- Example: A payroll system missing an essential tax calculation feature due to unclear specifications.

Bridging the Gaps

- **For User's Gap:** Conduct user research, improve UX design, and incorporate real-world feedback.
- **For Producer's Gap:** Strengthen requirement analysis, improve communication, and implement thorough testing.

Both gaps highlight the importance of aligning user needs with development goals through continuous quality assurance practices.

UNIT 2

1. Lifecycle of Software Testing

The **Software Testing Life Cycle (STLC)** is a structured approach for conducting testing systematically. It consists of the following phases:

1. **Requirement Analysis** – Understanding functional and non-functional requirements. Identifying testable requirements and potential risks.
2. **Test Planning** – Defining the test strategy, scope, resources, schedule, and tools. Risk assessment and budgeting are done here.
3. **Test Case Development** – Designing and writing test cases, test scripts, and preparing test data. Test scenarios are created based on requirements.
4. **Test Environment Setup** – Setting up hardware, software, networks, and test configurations. Ensuring compatibility with real-world use cases.
5. **Test Execution** – Running test cases, logging defects, and tracking them until resolution. Performing functional, performance, and security testing.
6. **Test Closure** – Preparing test reports, documenting lessons learned, and archiving test cases for future reference.

Importance: Ensures a structured and efficient approach to testing, improving software quality.

2. Fundamental Test Process

The **fundamental test process** consists of five main activities that guide effective software testing:

1. **Test Planning & Control** – Establish test objectives, define test scope, estimate resources, and monitor test progress.
2. **Test Analysis & Design** – Identify test conditions, design test cases, and set up traceability between requirements and tests.
3. **Test Implementation & Execution** – Prepare test scripts, execute test cases, report defects, and conduct regression testing.
4. **Evaluating Exit Criteria & Reporting** – Assess if testing objectives are met, measure defect density, and generate final test reports.
5. **Test Closure** – Final documentation, lessons learned, test case archiving, and preparing for future testing.

Purpose: Ensures a systematic testing process to detect and eliminate defects.

3. Five Principles of Software Testing

Software testing follows key principles that improve efficiency and effectiveness:

1. **Testing shows the presence of defects, not their absence** – Testing detects bugs but cannot guarantee defect-free software.
2. **Exhaustive testing is impossible** – It is impractical to test all inputs and scenarios; instead, focus on risk-based testing.
3. **Early testing saves time and cost** – Detecting and fixing defects in early stages (requirement/design) reduces rework costs.
4. **Defects cluster together** – A small number of modules contain most defects, requiring focused testing (Pareto Principle: 80% of defects are in 20% of modules).
5. **Testing is context-dependent** – Testing approach varies based on project type (e.g., banking software vs. gaming software).

Benefits: Helps optimize testing efforts and improve software reliability.

4. Verification vs. Validation

Verification and validation are two key software testing techniques that ensure software quality.

Feature	Verification	Validation
Definition	Ensures software meets design specifications and requirements.	Ensures software meets user needs and expectations.
Focus	Process-oriented – checks whether development activities are done correctly.	Product-oriented – checks whether the final product works as expected.
Methods	Reviews, walkthroughs, requirement analysis, design inspections.	Functional testing, system testing, user acceptance testing (UAT).
Timing	Performed before coding, during early development stages.	Performed after coding, during or after software implementation.
Performed By	Developers, business analysts, quality assurance (QA) team.	Testing team, end-users, QA engineers.
Example	Reviewing software design documents to check if they align with requirements.	Executing test cases to verify if login functionality works as expected.

Key Difference:

- **Verification** prevents defects by ensuring proper processes.
- **Validation** detects defects in the final product.

5. Gray Box Testing

Gray box testing is a hybrid approach that combines **black-box** and **white-box** testing. The tester has partial knowledge of the system's internal workings (e.g., database structures, algorithms) but tests it externally.

Advantages:

- Helps identify security vulnerabilities.

- More effective than black-box testing in finding hidden defects.
- Provides a balance between internal logic and real-world functionality.
- Reduces testing time compared to white-box testing.

Disadvantages:

- Requires partial knowledge of the internal system, which may not always be available.
- Complex test cases due to a mix of functional and structural testing.
- Not as thorough as full white-box testing in analyzing source code.

Example: Testing an e-commerce system while having knowledge of database schema but not full access to the code.

6. Defects and Their Categories

A **defect** is an error, bug, or flaw in a software application that causes it to function incorrectly or produce unintended results.

Categories of Defects:

1. **Functional Defects** – Errors in software behavior that do not meet functional requirements.
 - **Example:** A login system allows access with the wrong password.
2. **Performance Defects** – Issues related to software speed, responsiveness, or resource usage.
 - **Example:** A website taking more than 10 seconds to load.
3. **Usability Defects** – Poor user interface (UI) or experience (UX) design that affects accessibility.
 - **Example:** A button with unclear labeling or incorrect placement.
4. **Security Defects** – Vulnerabilities that allow unauthorized access, data breaches, or cyberattacks.
 - **Example:** A banking app exposing user account details due to weak encryption.
5. **Compatibility Defects** – Software working differently across platforms, devices, or browsers.
 - **Example:** A mobile app crashing only on iPhones but working fine on Android.

7. Types of Prototyping Software Development Models

Prototyping is a development approach where an early model of the software is created to gather feedback before full development.

Types of Prototyping Models:

1. **Throwaway (Rapid) Prototyping** – A temporary model is built for requirement clarification and discarded after feedback.
 - *Example:* UI mockups for a mobile app.
2. **Evolutionary Prototyping** – The prototype is continuously refined and eventually becomes the final product.
 - *Example:* Developing a web application in iterative stages.
3. **Incremental Prototyping** – Different components are developed separately and later integrated.
 - *Example:* A banking system where account management and transactions are built separately.
4. **Extreme Prototyping** – Used for web applications in three stages: static UI, functional UI, and backend integration.

- *Example:* A SaaS dashboard developed in phases.

Benefits: Helps gather early feedback, reduces risks, and improves software usability.

Unit 3

1. Boundary Value Testing

Boundary Value Testing (BVT) is a black-box testing technique used to identify defects at the boundary values of input domains. It is based on the principle that errors are more likely to occur at the edges rather than in the middle of an input range. Instead of testing all possible values, BVT focuses on minimum, maximum, just above, just below, and the exact boundary values.

Example:

Consider a form that accepts a user's age between **18 and 60**. The boundary test cases would be:

- **Valid Boundary Values:** 18, 60 (minimum and maximum valid values)
- **Just Inside Boundaries:** 19, 59 (values within the valid range)
- **Just Outside Boundaries:** 17, 61 (invalid values slightly outside the range)

Testing these values helps in detecting errors related to incorrect boundary conditions in the system.

2. Comparison of Weak Normal, Weak Robust, Strong Normal, and Strong Robust Equivalence Class Testing

Equivalence Class Testing divides input data into partitions where all values in a class are expected to produce similar results. It reduces the number of test cases while maintaining good coverage.

Testing Type	Valid Partitions	Invalid Partitions	Test Case Selection
Weak Normal	One value per valid class	None	Tests only valid cases
Strong Normal	All combinations of valid values	None	Covers all valid combinations
Weak Robust	One value per valid class	One value per invalid class	Tests both valid and invalid partitions
Strong Robust	All combinations of valid values	All combinations of invalid values	Most comprehensive, but requires more test cases

Example:

Consider an input form with two fields:

- **Age (18-60)**
- **Salary (10,000 - 50,000)**

For **Weak Normal** testing, we pick **one** value from each valid range, e.g., (25, 30,000).

For **Strong Normal**, we test all combinations like (18, 10,000), (60, 50,000), etc.

Weak Robust includes one invalid value per category, e.g., (17, 30,000).

Strong Robust tests all invalid combinations like (17, 5,000) and (65, 60,000).

3. Decision Table-Based Testing

Decision table-based testing is used for systems with multiple conditions leading to different actions. It helps in ensuring that all possible decision combinations are tested.

Example:

A bank system allows loan approval based on two conditions:

- 1. Credit Score ≥ 700
- 2. Annual Income $\geq 50,000$

Credit Score ≥ 700 Income $\geq 50,000$ Loan Approved?

Yes	Yes	Yes
Yes	No	No
No	Yes	No
No	No	No

This table ensures all combinations of inputs are covered, helping to detect any missing or incorrect logic.

4. DD Path and Nodes in Program Graph

A **Decision-to-Decision (DD) Path** is a sequence of statements in a program where control flows linearly without any decision-making nodes (e.g., if-else, loops). It represents the path between two decision points in the control flow graph of a program.

Types of Nodes in a Program Graph:

- 1. **Entry Node:** The start of the program.
- 2. **Process Node:** A statement that performs operations.
- 3. **Decision Node:** A condition-based branching point.
- 4. **Merge Node:** Where multiple branches converge.
- 5. **Exit Node:** The end of the program.

Example Program:

```
c
CopyEdit
int check(int x) {
    if (x > 10) // Decision Node
        x = x + 5; // Process Node
    else
        x = x - 5; // Process Node
    return x; // Exit Node
}
```

Here, the DD paths are:

1. Entry → Decision
 2. Decision → Process 1 → Exit
 3. Decision → Process 2 → Exit
-

5. Cyclomatic Complexity and McCabe's Basic Path Method

Cyclomatic Complexity (CC) measures the complexity of a program using its control flow graph. It helps in determining the minimum number of test cases required for full path coverage.

Formula:

$$CC = E - N + 2P$$

Where:

- **E** = Number of edges in the control flow graph
- **N** = Number of nodes
- **P** = Number of connected components (typically 1)

Example:

Consider this program:

```
c
CopyEdit
if (x > 0)
    x = x + 1;
else
    x = x - 1;
return x;
```

Control Flow Graph:

- Edges (E) = 4
- Nodes (N) = 3
- Components (P) = 1

$$CC = 4 - 3 + 2(1) = 3$$

This means at least 3 test cases are needed for complete path coverage.

McCabe's Basic Path Method uses CC to identify independent paths for testing, ensuring that all branches in the code are tested at least once.

6. Define/Use Testing (DU Testing) and DU/DC Paths

Define/Use Testing (DU Testing) verifies that variables are correctly defined and used in a program. It tracks how variables are assigned (definition) and later referenced (use).

Types of Paths:

1. **DU Path (Define-Use Path):** Tracks how a variable moves from being assigned a value to its usage.

2. **DC Path (Define-Clear Path):** Tracks whether a variable is cleared (reassigned) before reuse.

Example:

c

CopyEdit

```
int a = 5; // Define
```

```
a = a + 1; // Use
```

```
print(a); // Use
```

- The DU path ensures a is properly assigned before use.
 - The DC path checks if a is cleared before being reassigned.
-

7. Slice-Based Testing

Slice-Based Testing divides a program into segments (slices) based on variables and execution paths. It helps in debugging and isolating errors.

Example:

Consider the following program:

c

CopyEdit

```
int sum(int x, int y) {
```

```
    int z = x + y;
```

```
    int result = z * 2;
```

```
    return result;
```

```
}
```

If we analyze z, the relevant slice includes:

c

CopyEdit

```
int z = x + y;
```

```
int result = z * 2;
```

Testing only this slice ensures correctness while reducing complexity.

Unit 4

1. Types of Reviews Based on the Development Life Cycle Stage

Reviews are conducted at different stages to ensure the correctness and quality of software artifacts. The key types include:

- **Requirement Reviews:** Conducted during the requirement phase to ensure clarity, completeness, and feasibility.
- **Design Reviews:** Performed in the design phase to validate architecture, data flow, and system interactions.

- **Code Reviews:** Conducted during the coding phase to check adherence to coding standards, logic, and efficiency.
 - **Test Plan & Case Reviews:** Ensures test cases cover all requirements before testing begins.
 - **Release Reviews:** Performed before deployment to ensure system readiness and compliance with requirements.
 -
-

2. Types of Reviews in Verification

Verification ensures that software artifacts (like requirements, design, and code) meet specifications before execution. Common review types include:

a) Walkthroughs

- Informal review led by the author (developer, analyst, or tester).
- The author presents the document/code, and team members provide feedback.
- Helps improve understanding and identify basic issues.

b) Peer Reviews

- One developer reviews another's code or documents to find logical errors and maintain code quality.
- Ensures consistency with best practices and coding standards.

c) Inspections

- A **formal, structured review** where defects are documented and corrective actions are planned.
- Involves a **moderator, author, reviewers**, and a **scribe** for recording defects.
- Used for critical documents like requirements, designs, and security-sensitive code.

d) Technical Reviews

- Focuses on technical aspects such as **architecture, design patterns**, and **performance optimizations**.
 - Helps catch design flaws that could affect scalability, security, and maintainability.
-

3. Differences Between Beta and Alpha Testing

Feature	Alpha Testing	Beta Testing
Conducted By	Internal QA team, developers	Real users or selected customers
Environment	Simulated lab environment	Real-world environment
Objective	Detect critical bugs, usability issues	Get user feedback, check performance in live usage
Phase	Before Beta Testing	Before public release
Scope	Functional, UI, performance testing	Usability, reliability, and compatibility testing
Duration	Weeks to months	Few weeks

Feature	Alpha Testing	Beta Testing
Examples	Internal testing of mobile apps, enterprise software	Early access versions of apps, games, or web services

4. V-Model for Software & Its Pros and Cons

The **V-Model (Verification and Validation Model)** is a structured SDLC model where testing is conducted in parallel with development.

Stages of the V-Model

1. **Requirement Analysis** → Corresponds to **Acceptance Testing**
2. **System Design** → Corresponds to **System Testing**
3. **High-Level Design (HLD)** → Corresponds to **Integration Testing**
4. **Low-Level Design (LLD)** → Corresponds to **Unit Testing**
5. **Implementation (Coding Phase)**

Each development phase has a corresponding testing phase, ensuring defects are detected early.

Pros of the V-Model

- ✓ **Early defect detection:** Since testing runs in parallel with development, issues are identified sooner.
- ✓ **Clear structure:** Each phase has well-defined inputs and outputs.
- ✓ **Better documentation:** Ideal for projects requiring detailed documentation.

Cons of the V-Model

- ✗ **Rigid & inflexible:** No room for changes once development starts.
 - ✗ **High initial effort:** Requires detailed documentation and upfront planning.
 - ✗ **Not suitable for complex/iterative projects:** Lacks adaptability compared to Agile.
-

5. V&V Model of Testing

The **V&V Model (Verification & Validation Model)** ensures software correctness through:

- **Verification (Are we building the product right?)**
 - Ensures software meets specifications without executing it.
 - Methods: **Reviews, walkthroughs, inspections.**
- **Validation (Are we building the right product?)**
 - Ensures the software meets user needs through actual execution.
 - Methods: **Functional, usability, and performance testing.**

V&V Model vs. V-Model

- The **V-Model** strictly follows a **step-by-step development & testing** approach.
 - The **V&V Model** focuses on **continuous quality assurance** throughout the SDLC.
-

6. Roles & Responsibilities of the Development Group

The **development team** is responsible for designing, building, testing, and maintaining software.

Key Roles & Responsibilities

1. Software Developers

- Write, debug, and maintain code following best practices.
- Implement features based on design and requirements.
- Conduct **unit testing** and participate in **code reviews**.

2. Software Architects

- Design **high-level system architecture** and ensure scalability.
- Choose appropriate technologies and frameworks.
- Address security, performance, and integration concerns.

3. Quality Assurance (QA) Engineers

- Develop **test plans, cases, and automation scripts**.
- Perform functional, regression, and performance testing.
- Identify and document defects for resolution.

4. DevOps Engineers

- Manage **CI/CD pipelines**, automate deployments, and monitor system performance.
- Ensure **server stability, security, and infrastructure management**.

5. Business Analysts (BA)

- Gather requirements, analyze business needs, and define project scope.
- Bridge the gap between stakeholders and the technical team.

UNIT 5

1. What is Integration Testing? Explain the Big Bang Approach.

Integration Testing is a level of software testing where individual modules or components are combined and tested as a group. The goal is to ensure that data flows correctly between modules and that they work together as expected. It focuses on interface defects, communication issues, and data exchange problems.

Big Bang Approach:

- In this approach, all modules are integrated simultaneously after unit testing, and the entire system is tested as a whole.
- It does not follow an incremental process; instead, testing is performed only after all components are available.
- The main advantage is that it simulates the final product early. However, it also has significant drawbacks:
 - **Difficult debugging** – Since all modules are tested at once, pinpointing the root cause of defects is challenging.
 - **Late defect detection** – Bugs are found at a later stage, which may delay the project.
 - **Complexity** – The overall system is large, making testing cumbersome and time-consuming.

Big Bang Integration is suitable for smaller applications with fewer modules but is rarely used in large-scale projects due to its inefficiency.

2. Bottom-Up, Top-Down, and Sandwich Approaches in Integration Testing

1. Bottom-Up Approach:

- Testing starts with lower-level modules (subsystems) and progresses upward to higher-level modules.
- Uses *drivers* (dummy programs) to simulate higher-level modules that are not yet developed.
- Once lower modules are tested and integrated, the higher-level modules are introduced.

Advantages:

- Defects in lower-level modules are detected early.
- Helps validate the core functionalities before higher modules depend on them.

Disadvantages:

- Higher-level modules are tested late, which may delay the overall testing process.
- Requires drivers, which increase development effort.

2. Top-Down Approach:

- Testing begins with higher-level modules, and lower-level modules are integrated progressively.
- Uses *stubs* (dummy modules) to simulate lower-level modules that are not yet available.

Advantages:

- Major design flaws are detected early since testing starts from the top.
- Stubs can be simple, making initial testing easier.

Disadvantages:

- Requires stubs, which add extra development effort.
- Lower-level module defects may go unnoticed initially.

3. Sandwich Approach (Hybrid Approach):

- A combination of both Bottom-Up and Top-Down approaches.
- System is divided into different layers, and both approaches are applied simultaneously.
- Middle-layer modules are tested last.

Advantages:

- Balances the limitations of Top-Down and Bottom-Up testing.
- Both lower-level and higher-level modules are tested early.

Disadvantages:

- More complex to manage and requires both drivers and stubs.
- Might increase the overall testing effort.

-

3. Performance Testing, Volume Testing, and Stress Testing

1. Performance Testing:

- Evaluates the system's speed, responsiveness, and stability under a specific workload.
- Measures parameters like response time, throughput, and resource usage.
- Helps identify bottlenecks and optimize performance.

Example: Checking how fast a website loads under normal traffic conditions.

2. Volume Testing:

- Tests the system's ability to handle large amounts of data.
- Evaluates the performance when the database grows significantly.
- Helps ensure that the application does not crash due to data overload.

Example: Testing an e-commerce site with millions of product entries.

3. Stress Testing:

- Assesses the system's behavior under extreme conditions, such as high traffic or resource exhaustion.
- Identifies failure points and how the system recovers from crashes.

Example: Simulating thousands of users accessing a banking app simultaneously to test stability.

4. Commercial Off-The-Shelf (COTS) Software Testing and Its Features

What is COTS Software?

- COTS (Commercial Off-The-Shelf) software refers to pre-built, ready-made software solutions available for commercial use.
- Examples: Microsoft Office, Adobe Photoshop, SAP ERP.

COTS Testing:

- Ensures that the software meets business requirements and integrates well with other systems.
- Focuses on compatibility, security, and functional validation.

Features of COTS Testing:

1. **Predefined Functionality** – Since COTS products are pre-built, testing focuses on verifying if they meet business needs.
2. **Compatibility Testing** – Ensures seamless integration with existing applications.
3. **Security Testing** – Identifies vulnerabilities since COTS solutions are widely used.
4. **Limited Customization** – Unlike custom-built software, COTS has limited flexibility, making testing important for user-specific adaptations.

5. 5. Difference Between Load Testing and Stress Testing

Aspect	Load Testing	Stress Testing
Purpose	Checks system performance under expected workload.	Evaluates system behavior under extreme, beyond-normal conditions.
Goal	Measures response time, throughput, resource usage, and system stability under typical user loads.	Identifies breaking points, system failures, and recovery mechanisms when overloaded.
Test Scenario	Simulates real-world usage with a normal or increasing number of users.	Pushes the system to its limits with excessive traffic, requests, or resource exhaustion.
Focus	Ensures system can handle peak loads efficiently.	Tests how the system behaves when overloaded and whether it recovers gracefully.
Example	Testing an e-commerce website with 5,000 concurrent users to check response time.	Flooding a banking system with 50x its usual traffic to see if it crashes.
Outcome	Helps optimize system performance under expected load conditions.	Identifies failure points, system crashes, and performance bottlenecks under extreme conditions.

6. Code Review & Unit Testing Process

Code Review Process:

A systematic examination of source code to find defects and improve quality. Steps include:

1. **Preparation:** Developers prepare the code and documentation for review.
 2. **Inspection:** Reviewers examine the code for logic errors, coding standards, and security issues.
 3. **Discussion & Feedback:** Issues are discussed, and improvements are suggested.
 4. **Refinement:** Developers make necessary corrections and update the code.
 5. **Approval & Merge:** Once issues are resolved, the code is merged into the main branch.
-

Unit Testing Process:

Unit testing verifies the functionality of individual components or modules. Steps include:

1. **Test Case Design:** Identify test inputs, expected outputs, and test conditions.
2. **Execution:** Run the test cases using unit testing frameworks (e.g., JUnit for Java, pytest for Python).
3. **Validation:** Compare actual results with expected outcomes.
4. **Debugging & Fixing:** If a test fails, debug the module and correct errors.
5. **Re-testing:** Execute the test again to ensure issues are resolved.

Benefits:

- Detects defects early in development.
- Improves code maintainability and reliability.
- Reduces the risk of bugs propagating to later stages.

